# Editorial

Tasks, test data and solutions were prepared by: Fran Babić, Toni Brajko, Jakov Celin, Dorijan Lendvaj, and Patrik Pavić.

Implementation examples are given in attached source code files.

## Task Wow

Prepared by: Dorijan Lendvaj
Necessary skills: strings

For simpler implementation, it can be observed that it is sufficient to consider only the first row of input to determine each letter. It is known that the first character of the upper row of a letter will always be `'\'`. If the letter is `'w'`, then the 5th character of the upper row will also be `'\'` (if it exists). Otherwise, the corresponding letter is `'v'`.

Based on the decision of which letter is currently appropriate, we move the *pointer* accordingly. For more details, refer to the official solution implementation.

## Task Zid

Prepared by: Patrik Pavić
Necessary skills: two pointers

Let's fix the top and bottom row of the rectangle, $l$ and $r$. Let $x_j$ be the number of nails driven into the $j$-th column when considering only the rows from $l$ to $r$. The value of $x_j$ can be computed using prefix sums. Specifically, if we denote $p_{ij}$ as the number of nails in the $j$-th column and all rows $\leq i$, then we have

$$x_j = p_{rj} - p_{(l-1)j}$$

If the left and right columns of the image are $a$ and $b$, then the rectangle $[l, r] \times [a, b]$ can contain at most one nail. This condition is equivalent to

$$\sum_{j=a}^{b} x_j \leq 1$$

Thus, we need to count how many subarrays of the sequence $x_j$ have a sum of at most 1. This can be computed using the two pointers technique in $O(m)$ complexity.

The overall complexity of the algorithm is $\mathcal{O}(n^2 \cdot m)$.

## Task Tornjevi

Prepared by: Fran Babić
Necessary skills: *greatest common divisor* function, ad-hoc

For a set of positive integers $S$, we can observe that $\gcd(S) \leq \min(S)$, i.e., $\gcd(S) \leq m$ for every $m \in S$. Therefore, for every $l \leq i \leq r$, it holds that $\gcd(h_l, h_{l+1}, \ldots, h_r) \leq h_i$, and we conclude that $h_i$ is good in an interval if and only if it divides all numbers in that interval. We can then observe that the $i$-th tower is good in the interval $[l, r]$, where $l \leq i \leq r$, if and only if it is good in the intervals $[l, i]$ and $[i, r]$.

For an element in the sequence with index $i$, it is sufficient to independently determine the smallest left boundary $l_i$ that satisfies the condition that tower $i$ is good in the interval $[l_i, i]$ and the largest right

boundary $r_i$ that satisfies a similar condition. Since these two problems are symmetric, we describe the algorithm for determining the boundary $l_i$.

The idea of the algorithm is to maintain the *change indices*, i.e., the set of indices $t$ such that $\gcd(h_t, \ldots, h_i) \neq \gcd(h_{t-1}, h_t, \ldots, h_i)$. Since it holds that $\gcd(h_l, \ldots, h_r)$ divides $\gcd(h_{l+1}, \ldots, h_r)$ (analogously, $\gcd(h_l, \ldots, h_r)$ divides $\gcd(h_l, \ldots, h_{r-1})$), the size of this set can be at most $\log_2 h_i$, because after each change index, the greatest common divisor will be at least halved. Additionally, it is easy to see that the set of change indices for index $i+1$ will be a subset of the set of change indices for $i$ (except for index $i+1$). Thus, it is sufficient to maintain this set of changes for each $i$, and the largest index in this set will be the desired boundary $l_i$. The complexity of this algorithm is $O(n \log^2 n)$. Details of the implementation can be found in the attached source code.

## Task Crtež

Prepared by: Jakov Celin
Necessary skills: segment tree, fast exponentiation, compression

The author made an error when writing the problem statement for Crtež. Specifically, the author intended to define the problem such that, in the second case, the coloring should stop when encountering a value **different** from 0. The test cases were created according to the intended problem statement, without the error in the text, which resulted in all contestants receiving 0 points for this problem. We sincerely apologize to all contestants for this oversight.

Below is the problem editorial, where in the second case, the coloring stops when encountering a value different from 0.

Let us consider an arbitrary sequence after completing all coloring operations.

**Observation 1.** Each color $X > 0$ will cover some interval of positions in the final sequence.

*Proof.* Notice that each painting operation with color $X > 0$ affects an interval of sequence positions whose values are 0. Painting with color $-1$ affects adjacent positions in the sequence whose values are 0, so Observation 1 follows directly. □

**Observation 2.** Each game is equivalent to the game obtained by sorting the intervals by their left/right boundary and replacing their values with their position in the sequence after sorting.

*Proof.* Sort the intervals by their left boundaries. Each color $X > 0$ covering some interval of the sequence will map to a color $Y > 0$ that represents the position of the interval in the sorted order. We observe that each $Y > 0$ is paired with a unique $X > 0$, meaning the mapping is bijective, so all game equivalence conditions are satisfied. □

**Observation 3.** Fix the final sequence after an arbitrary number of operations and replace it with the equivalent sequence from Observation 2. We can always achieve this by first performing all operations that paint fields with -1 and then performing operations on the intervals from left to right.

*Proof.* If we painted a field with -1, the painting condition states that this field was 0 before the operation, meaning no interval of colors greater than 0 contained this position. Thus, we can perform the painting operation to $-1$ at any point before the current one, i.e., at the very beginning of the painting operations. Painting an interval with color $X > 0$ stops when we exit the sequence, encounter a -1, or reach the right boundary of an interval already painted. If we paint the sequence in the order given by Observation 3., all these properties are preserved, meaning that performing the operations leads to a fixed sequence. □

**Observation 4.** If $N$ is the number of free fields(with value 0) in the sequence before any painting, then the number of different games that can be played with an arbitrary number of operations such that there is no pair of equivalent games among them is exactly $3^N$.

*Proof.* Fix the number of occurrences of $-1$ among the free fields after an arbitrary number of operations where we end with a game. Two games with different numbers of $-1$ occurrences in their final sequences are never equivalent, so we can count the number of different games separately for each number of $-1$ occurrences in the sequence and sum them up. Let $X$ be the number of $-1$ occurrences in the sequence. The number of ways to place them in $N$ positions is $\binom{N}{X}$. Consider the remaining fields ($N - X$ of them). From Observation 3, we can fix which positions are the right boundaries of the intervals, and then the sequence is uniquely determined (by painting from left to right). For a fixed number of $-1$ occurrences in the sequence, the number of different games is $\binom{N}{X} \cdot 2^{N-X}$. Summing over the number of $-1$ occurrences directly gives the binomial theorem formula:

$$\sum_{X=0}^{N} \binom{N}{X} \cdot 2^{N-X} = (2+1)^N = 3^N$$

□

From the first four observations, it follows that at any moment, we are interested in the number of occurrences of 0 in the sequence and need to output $3^{(\text{occurrences of 0 in the sequence})}$.

The first subtask is solved by directly iterating through the sequence positions, changing 0 to $-1$ and vice versa, and counting the number of 0s in the sequence. The time complexity of the algorithm is $O(NQ)$, and the memory complexity is $O(N)$.

The second subtask is solved using a segment tree with propagations and fast exponentiation. Each node in the segment tree stores the size of the interval it covers, the number of 0s in it, and whether it needs to be flipped or not. Using propagations, we can easily flip all 0s to $-1$ and all $-1$s to 0 using stored values in each node. After each change in the sequence, we can also determine the number of 0s in the entire sequence using the segment tree. Calculating $3^{(\text{number of 0s in the sequence})}$ should not be done naively, as it would keep the code complexity the same, so we use fast exponentiation for this. The time complexity of the algorithm is $O(Q \log N)$, and the memory complexity is $O(N)$.

Achieving full points on the problem was possible using a more advanced data structure called sparse segment tree, but its knowledge was not required. First, process all $Q$ intervals and then compress them, i.e., divide the sequence of length $N$ into $O(Q)$ intervals where all positions within each of the $O(Q)$ intervals have the same value. We can now build a segment tree and use the solution from the second subtask. The time complexity of the algorithm is $O(Q \log Q + Q \log N)$, and the memory complexity is $O(Q)$.

## Task Stablo II

Prepared by: Toni Brajko and Patrik Pavić
Necessary skills: union-find

There are multiple solutions to this problem, but here we will describe a simple one.

We will solve the problem *in reverse*. We will color the edges with colors from the $k$-th to the first, and once we color an edge, we will not recolor it. Through this process, we will obtain the final coloring of the tree.

The key idea is that on a given path, we do not traverse edges that have already been colored, only those that have not. We want to efficiently support two types of operations:

- color an edge

- for a given vertex on a path, find the next uncolored edge

We root the tree at vertex 1. Instead of finding the next edge on a path, it is sufficient to determine which is the next uncolored edge on the path from a given vertex to the root. For the vertices $u$ and $v$ from the query, let us find such edges $e_u$ and $e_v$. If they are different, we color the one that is farther from the root. We repeat this process until we get the same edge for both vertices. In this case, that edge is not on the path from $u$ to $v$, and we have already colored all the desired edges.

We can implement this using the *union-find* structure. When we color an edge, we merge the vertices that the edge connects into the same component (we do not change the tree but maintain information about each vertex's component). If we want to find the next uncolored edge for some vertex $u$, we will find the vertex $v$ with the smallest depth in the same component as $u$. The required edge will be the one connecting $v$ with its parent. We know that $v$ and its parent are not in the same component, which means that the edge between them is uncolored. By coloring that edge, the vertex with the smallest depth in that component will no longer be $v$.

Each edge will be colored at most once, and since the total number of edges is $n-1$, we will have $\mathcal{O}(n)$ merges and component searches. The overall complexity of the algorithm is $\mathcal{O}(n \cdot \alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function related to the complexity of the *union-find* structure.